System Design Interview Summary

Problem: Design a Truecaller-like Caller ID & Spam Detection App | Mock Interview Quick-Reference

Candidate track	Senior Software Engineer (Java / Spring Boot)
Problem	Global Caller ID + Crowdsourced Spam/Scam Detection
Interview duration	45-60 minutes (Google-style RCTFC format)
Overall score	5.5 / 10 (Lean No / Weak Hire trending to Hire with more practice)

1. Requirements

Functional Requirements

- Global search: lookup a phone number and return caller name/ID
- Identify who an unknown incoming call belongs to
- Detect and flag spam, scam, and telemarketing calls

Non-Functional Requirements

- Low-latency search (single-digit to low double-digit ms)
- Support 100M+ daily active users
- High availability: 99.99%
- Elastic/dynamic scalability under load

2. Capacity Estimation

Metric	Assumption	Result
DAU	-	100,000,000
Read (search) QPS	10 searches/user/day	~11,574/sec (baseline) -> ~23,148/sec (peak, 2x)
Write QPS	Read:Write = 100:1	~115/sec (baseline) -> ~230/sec (peak)
Raw contact storage	100M users x 500 contacts x 200 bytes	10 TB (undeduplicated, staging data)
Resolved index size	~1 entry per unique global phone number	Much smaller than 10TB - billions of rows, flat schema

Reminder: 1 TB = 10^12 bytes, 1 PB = 10^15 bytes - double check unit conversions under pressure.

3. Core Algorithm - Caller Name Resolution

Use **weighted whole-string frequency** (not token-level splitting) to pick the display name for a phone number. Token-level frequency risks stitching together a name nobody actually saved.

- **Trust score per contributor:** account age, phone-verification status, submission velocity, historical accuracy
- **Dedup** by device/account fingerprint so one physical user = one vote
- **Weighted frequency:** sum(trust_score) per name variant wins, not raw count - defends against botnets/fake uploads
- **Business verification override:** match against a verified business registry; verified name wins outright

Spam Detection (separate pipeline from naming)

- Explicit user reports, weighted by reporter trust, decayed over a rolling window (e.g. 30 days)
- Call-pattern heuristics (high volume, short duration, low pickup rate) - harder to game than user input
- Spam label assigned only past a confidence threshold combining both signal types

4. API Design

Endpoint	Method	Request	Response
/api/v1/search/{number}	GET	-	id, name, category, photo, displayName
/api/v1/spam/{number}	POST	category, reportedBy	status (success/failure)
/api/v1/contacts	POST	List<{number, name, reportedBy, photo}>	success message

Convention notes: nouns as resources, verbs via HTTP method; avoid duplicating the path key in the request body.

5. Data Model

MongoDB - Raw Contacts (write path)

One document per user; nested array of contacts. **Shard/partition key: userId** - matches the 'upload full phonebook' write pattern and distributes write load evenly.

Fields: _id/userId, uploadedAt, contacts[{number, savedName, photo, reportedCategory}]

DynamoDB - Resolved Index (read path)

One item per unique phone number. **Partition key: phoneNumber** - the only query field, guarantees O(1) exact-match lookup with even distribution.

Attribute	Type	Notes
phoneNumber (PK)	String	Globally unique, exact-match key
resolvedName	String	Winner from weighted-frequency algorithm
spamLabel	String	NONE / SPAM / SCAM / TELEMARKETER
spamScore	Number	Confidence score 0-1, decays over time
businessVerified	Boolean	True if matched to verified registry
photo	String	S3 URL
lastUpdated	Timestamp	Drives staleness checks / TTL logic

Principle: access pattern drives data model - flexible/nested for bulk offline write+read; flat/strict for low-latency point lookups.

6. Processing Pipeline

- **Spam reports (safety-critical, low volume):** API publishes event to Kafka -> consumer aggregates reports in a short window -> single batched write to DynamoDB -> active cache invalidation. Avoids hot-key write contention from bursty reports on one number.
- **Contact uploads / name resolution (high volume, less urgent):** Scheduled Apache Spark batch job reads MongoDB, groups by phoneNumber, computes weighted-frequency winner, writes bulk results to DynamoDB. Spark chosen over cron+worker pool for built-in distributed processing and fault tolerance over billions of records.

7. Caching Strategy

- **Pattern:** Redis cache-aside in front of DynamoDB
- **Eviction:** LRU - phone number lookups follow a Zipfian/power-law distribution (popular businesses/scam numbers queried far more than the long tail)
- **Invalidation:** Active invalidation on write (delete/update key immediately after DB write) + short TTL (e.g. 24h) as a safety net - spam labels are safety-critical and cannot wait on passive TTL expiry alone

8. Failure Scenarios & Mitigations

Scenario	Mitigation
Cache stampede / thundering herd (viral number, cache miss storm)	Request coalescing via lock (Redis SETNX) so only one request populates cache per key; others wa

Hot partition on DynamoDB (viral number gets millions of reads)	Redis cache absorbs most read load; DynamoDB adaptive capacity; consider caching layer TTL/jitter
Regional outage (DynamoDB region down)	DynamoDB Global Tables - multi-region active-active replication. Tradeoff: eventual consistency acro

9. Scorecard

Category	Score	Feedback
Requirement Gathering	6/10	Functional reqs came quickly; didn't probe data source/freshness unprompted
Scale Estimation	7/10	Clean QPS math once corrected; initial TB/PB unit error
API Design	7/10	Clean REST after 2 rounds of correction
Database Design	4/10	Needed full guidance on Mongo/DynamoDB schema
High-Level Architecture	6/10	Reasonable pipeline once guided, not self-derived
Component Design	5/10	Good instinct on frequency; needed guidance on anti-gaming defenses
Scalability	6/10	Understood partition keys; didn't proactively flag hot-key risk
Reliability/Availability	6/10	Correct instinct on CAP tradeoff; didn't know Global Tables by name
Caching	5/10	Cache-aside solid; eviction/invalidation/stampede needed guidance
Data Modeling	5/10	Understood the 'why' partially; articulation was muddled
Communication Skills	7/10	Honest when stuck rather than guessing wildly
Tradeoff Analysis	6/10	Applied tradeoffs when framed, didn't generate independently
Problem Solving	5/10	Relied on interviewer guidance for ~40% of deep dive
Overall	**5.5/10**	**Weak Hire trending to Hire with more focused practice**

10. Priority Study Topics Before Next Interview

- NoSQL data modeling - MongoDB document design, DynamoDB partition/sort key strategy (weakest area)
- Distributed systems failure patterns: cache stampede, hot partitions, circuit breakers, idempotency
- DynamoDB specifics: Global Tables, GSI/LSI, adaptive capacity
- Privacy/compliance in crowdsourced-PII systems: GDPR right-to-be-forgotten, consent for uploading others' contacts
- Rate limiting the search API itself (anti-scraping) - was not raised proactively
- Practice giving specific, justified answers instead of category-level answers (e.g. name a message queue, not just 'a queue')